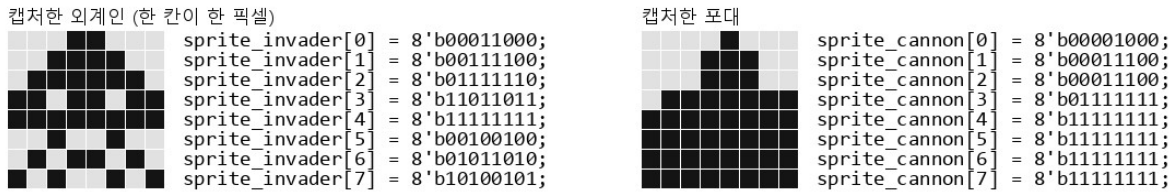


Q.10 실습: 스프라이트 시현 방식

Q.10 은 받은 Makefile 그대로 빌드되고 실행되었다. 캡처한 화면의 외계인과 포대를 픽셀 단위로 확대해 보니 Verilog 소스의 스프라이트 ROM 값과 정확히 일치했다.

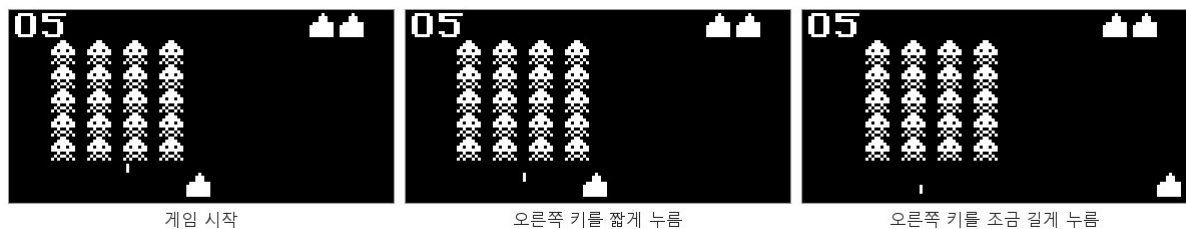


캡처한 화면과 스프라이트 ROM 값 비교

구분	아두이노 C++	베릴로그 RTL
그리는 방법	drawBitmap 함수로 그림 데이터를 프레임 버퍼에 한 번 복사한다	프레임 버퍼 없이, 화면을 훑는 좌표마다 ROM 에서 그 점의 값을 꺼낸다
외계인이 여러 마리일 때	반복문을 여러 번 실행한다. 시간이 더 든다	반복문이 비교 회로 여러 개로 복사된다. 면적이 더 든다

Q.11 실습: 좌우 화살표 키의 이상 반응

Q.11 은 SystemC 라이브러리 경로가 틀려 처음에 실행되지 않았고, Makefile 의 경로를 고쳐 실행했다. 오른쪽 키를 짧게 누르면 포대가 움직이지 않았고, 조금만 길게 눌러도 화면 끝까지 한 번에 튕겨 갔다.



Q.11 에서 오른쪽 키를 눌렀을 때의 반응

키를 누른 시각과 테스트벤치가 키 상태를 읽은 시각을 기록해 원인을 찾았다.

```

hj@WSL: ~/space_invader_by_Gemini/Q11_SC_Final
$ LD_PRELOAD=./kbspy.so ./obj_dir/Vspace_invaders_engine_glcd # SDL 호출 시각 기록
$ python3 timeline.py drive.log kbspy.log
4.303 s TB 폴링 L=0 R=0 F=0
4.369 s XTest 키 누름: Right
4.759 s SDL 이벤트 DOWN Right
4.759 s SDL 이벤트 UP Right
4.759 s TB 폴링 L=0 R=0 F=0
5.217 s TB 폴링 L=0 R=0 F=0
5.661 s TB 폴링 L=0 R=0 F=0
6.021 s XTest 키 누름: Right
6.102 s SDL 이벤트 DOWN Right
6.103 s TB 폴링 L=0 R=1 F=0
6.571 s TB 폴링 L=0 R=1 F=0
7.031 s SDL 이벤트 UP Right
7.032 s TB 폴링 L=0 R=0 F=0
7.459 s TB 폴링 L=0 R=0 F=0
7.623 s XTest 키 누름: Left
7.911 s SDL 이벤트 DOWN Left
7.912 s TB 폴링 L=1 R=0 F=0
8.344 s SDL 이벤트 UP Left
8.345 s TB 폴링 L=0 R=0 F=0

```

키 입력과 테스트벤치 폴링 시각 기록

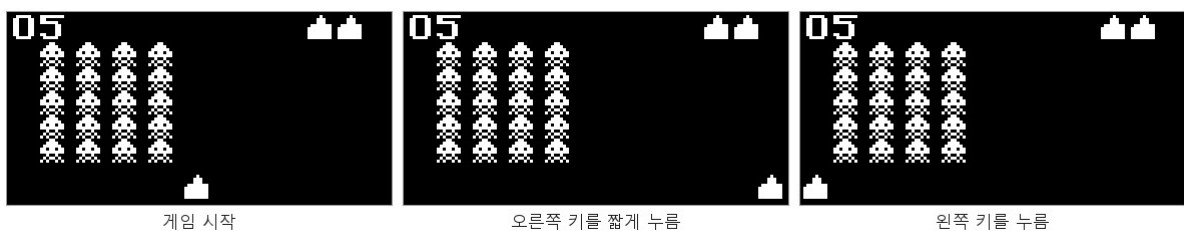
첫째, 테스트벤치는 시뮬레이션 시간으로 16.6 밀리초마다 키 상태를 읽는 폴링 방식인데, 시뮬레이션이 느려서 실제로는 약 0.45 초 간격이었다. 이보다 짧게 누른 키는 누름과 땀이 함께 처리되어 사라진다.

둘째, RTL 에서 포대 이동이 game_tick 과 묶여 있지 않아 키가 눌러 있는 동안 클릭마다 한 픽셀씩 움직인다. 그래서 키가 한 번만 읽혀도 포대가 화면 끝까지 간다.

Q.13 시뮬레이션 실습

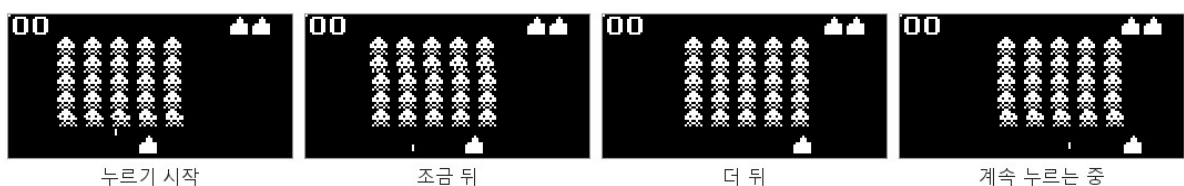
Q.13 테스트벤치는 키를 정해진 간격으로 읽지 않고, 키를 누르거나 땀 때마다 바로 처리하는 이벤트 방식으로 바뀌었다. 또 EMULATED_CO_SIM 설정 하나로 같은 테스트벤치를 FPGA 코에뮬레이션에도 쓸 수 있게 되었다.

새 테스트벤치에 원본 RTL 을 넣으면 짧은 키 입력도 잘 읽히지만, 그 한 번에 포대가 화면 끝까지 튕겨 간다. 테스트벤치만 고쳐서는 해결되지 않는다.

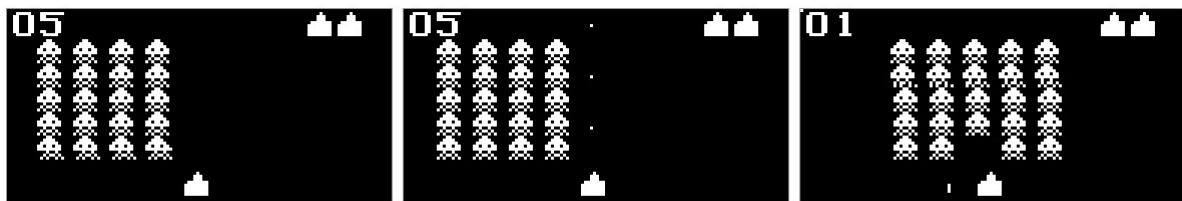


Q.13 테스트벤치와 원본 RTL

최종 RTL 에서는 키를 누르고 있는 동안 포대가 일정한 속도로 조금씩 움직였다. 발사 버튼도 원본은 한 번 누르는 사이 연사가 되고 총알이 보이지 않았지만, 최종본은 한 번에 한 발씩 보이며 날아갔다.



Q.13 테스트벤치와 최종 RTL



원본 RTL: 시작하자마자 연사

원본 RTL: 총알이 안 보임

최종 RTL: 한 발씩 보임

발사 버튼 비교

보완 내용	원본 RTL	최종 RTL
좌우 이동	클릭마다 움직인다	화면을 한 번 다 그린 뒤 키를 확인하는 상태 STATE_CHK_BTN 을 추가해 한 화면에 한 픽셀씩 움직인다
발사	누르고 있는 동안 계속 발사한다	버튼이 눌리는 순간만 잡아내는 에지 검출 신호 btn_fire_pe 로 한 발씩 쏜다
총알 이동	클릭마다 움직여 보이지 않는다	game_tick 이 올 때만 움직인다
포대 피격	데이터 블록에서 상태 레지스터를 직접 바꾼다	player_hit 신호만 올리고 상태 변경은 상태 머신이 맡는다

느낀 점

눈으로는 키가 잘 안 먹고 가끔 튼다는 느낌뿐이었지만, 시간을 재어 보니 테스트벤치의 느린 폴링과 RTL 의 너무 빠른 이동이라는 두 원인으로 나뉘었다. 같은 테스트벤치에 RTL 만 바꿔 넣은 비교로 RTL 도 고쳐야 한다는 것을 확인할 수 있었다.

중간 버전 하나는 두 always 블록이 같은 상태 레지스터를 바꾸는데, 시뮬레이션은 통과했지만 FPGA 합성에서 실패했다. 시뮬레이션 통과가 하드웨어로 만들 수 있다는 뜻은 아니었다.

이번 보완은 다음 단계의 준비이기도 했다. 코에뮬레이션에서는 클릭마다 PC 와 FPGA 가 통신해서 회로가 매우 느리게 돈다. 한 화면에 한 번 움직이기와 버튼이 눌리는 순간만 인식하기는 클릭 속도와 상관없이 게임이 똑같이 동작하게 해 준다.

내 칩 견적도 직접 확인했다. Gemini 는 약 4 제곱밀리미터를 예상했지만 Yosys 와 ETRI 표준 셀 라이브러리로 합성하니 셀 면적은 약 0.68 제곱밀리미터였다. AI 의 예상은 직접 측정해서 확인해야 한다. 또 Q.13 회로는 신호 핀이 20 개 필요한데, Q.14 는 좌표 대신 픽셀 값과 동기 신호만 보내 7 개로 줄이고 그림은 칩 밖의 마이크로컨트롤러가 그리게 했다. 칩 면적과 핀 수라는 실제 제약이 C++와 RTL 중 어디서 그릴지를 정하는 기준이 된다는 것을 배웠다.